RIO 정리

Oct 17, 2026 · @성준

1. 들어가며

IOCP는 이미 Windows에서 대규모 동시 접속을 처리하는 사실상 표준 기술이다. 하지만 초당 수십만~수백만 건의 I/O를 처리해야 하는 초고빈도 환경(HFT, 대규모 UDP 실시간 트래픽 등)에서는 IOCP조차 병목이 될 수 있다. RIO는 이 지점을 정면으로 겨냥해 Windows 8 / Server 2012부터 추가된 Winsock 확장 API다. 일반 Winsock 함수와 달리 `WSAIoctl`로 함수 포인터 테이블을 얻어 호출한다(7.1절). 이 글은 RIO가 어떤 문제를 해결하는지, 핵심 구성 요소가 무엇인지, IOCP와 어떻게 연결되는지, 그리고 실제 사용법과 예시를 정리한다.

2. IOCP만으로 부족해지는 지점

IOCP 기반 서버에서 `WSASend` / `WSARecv`를 호출할 때마다 다음 비용이 반복적으로 발생한다.

1. **시스템 콜(유저모드 ↔ 커널모드 전환) 비용**: 호출 1건마다 모드 전환이 발생한다. 전환 자체는 마이크로초 단위지만, 초당 수십만 건이 쌓이면 무시할 수 없는 CPU 비용이 된다.
2. **버퍼 검증/락(lock) 비용**: 커널은 매 I/O마다 유저모드 버퍼가 유효한 메모리인지 검증하고, I/O 도중 페이징되지 않도록 락을 건다. 이 과정이 호출마다 반복된다.
3. **완료 통지의 개별 처리**: `GetQueuedCompletionStatus()`는 한 번에 완료 항목 하나(또는 `GetQueuedCompletionStatusEx`로 여러 개)를 받지만, 여전히 통지 자체의 오버헤드가 호출 단위로 발생한다.

RIO는 이 세 가지를 각각 **배치 제출**, **버퍼 사전 등록**, **배치 완료 수거**로 줄인다.

3. RIO의 핵심 구성 요소

3.1 Registered Buffer — 버퍼를 미리 등록해 검증 비용을 없앤다

```
char* pBuffer = (char*)VirtualAlloc(NULL, bufferSize, MEM_RESERVE |
MEM_COMMIT, PAGE_READWRITE);
RIO_BUFFERID bufferId = g_RIORegisterBuffer(pBuffer, (DWORD)bufferSize);
```

한 번 등록해두면, 이후 이 메모리 영역을 사용하는 모든 I/O는 매번 커널에 버퍼 유효성을 검증받을 필요가 없다. 실제 송수신 시에는 등록된 버퍼 안에서 오프셋과 길이만 지정한 `RIO_BUF`를 넘긴다.

```
RIO_BUF rioBuf;
rioBuf.BufferId = bufferId;
rioBuf.Offset   = offsetInBuffer;
rioBuf.Length   = dataLength;
```

`RIO_BUF`는 하나의 버퍼 등록 안에 완전히 포함되는 연속 구간이어야 하며, 등록한 크기보다 큰 값을 지정한다고 등록 범위가 늘어나지는 않는다. 버퍼를 등록하면 해당 가상 메모리 페이지가 물리 메모리에 고정(lock)된다. 작은 버퍼를 여러 개 따로 등록하면 등록마다 페이지 단위로 물리 메모리를 차지하고 등록 자체에도 소량의 메모리가 들기 때문에, 문서는 여러 할당을 하나의 큰 할당으로 모아 한 번에 등록하라고 권한다. 큰 메모리 블록 하나를 등록해두고, 그 안을 여러 세션이 슬롯 단위로 나눠 쓰는 방식(슬롯 할당자)이 일반적인 이유다.

> **문서의 서술 불일치**: `RIOSend` 문서의 비고에는 송신 데이터가 등록 버퍼의 일부든 전체든 참조하면 그 송신이 끝날 때까지 등록 버퍼 전체를 사용하지 않아야 하고, 여기에는 그 버퍼의 다른 부분을 수신이나 다른 송신에 쓰는 것도 포함된다는 문장이 있다. 반면 `RIO_BUF` 문서는 큰 버퍼 하나를 등록하고 그 조각(buffer slice)을 필요할 때마다 쓰는 사용법을 성능 면에서 유리한 방식으로 설명한다. 문자 그대로 읽으면 두 서술은 양립하지 않는다. 이 글은 `RIO_BUF` 문서가 설명하는 사용 방식(하나의 등록을 슬롯으로 분할)을 따르고, 같은 슬롯을 두 요청이 동시에 쓰지 않는 규칙을 최소한으로 지킨다. `RIOSend` 문구를 엄격하게 따라야 하는 환경이라면 송신용과 수신용 등록을 분리하거나 세션·워커 단위로 등록을 나누는 설계를 검토하고, 대상 환경에서 직접 검증한다. 슬롯 할당/반납은 애플리케이션이 직접 관리해야 하며, 이 관리 로직의 정확성이 RIO 구현 난이도의 상당 부분을 차지한다.

3.2 Request Queue(RQ)와 Completion Queue(CQ)

- **RQ**: 소켓 하나당 하나씩 만든다. `RIOCreateRequestQueue()`로 생성하며, 이 소켓에 대한 송수신 요청이 여기에 쌓인다.
- **CQ**: 완료된 I/O 결과가 모이는 곳. `RIOCreateCompletionQueue()`로 생성하며, **여러 개의 RQ(즉 여러 소켓)가 하나의 CQ를 공유**할 수 있다.

```
RIO_NOTIFICATION_COMPLETION notifyType;
notifyType.Type = RIO_IOCP_COMPLETION;
notifyType.Iocp.IocpHandle = hIOCP;
notifyType.Iocp.CompletionKey = (void*)cqCompletionKey;
notifyType.Iocp.Overlapped = pOverlappedForCq;

RIO_CQ cq = g_RIOCreateCompletionQueue(queueSize, &notifyType);

RIO_RQ rq = g_RIOCreateRequestQueue(
    socket,
    maxOutstandingReceive, maxReceiveDataBuffers,
    maxOutstandingSend, maxSendDataBuffers,
    cq,  // Receive CQ
    cq,  // Send CQ (다르게 분리할 수도 있음)
    (void*)pSessionContext
);
```

N개의 커넥션을 소수의 CQ로 묶어 처리할 수 있다는 점이 확장성의 핵심이다. CQ 하나를 워커 스레드 하나가 전담하는 1:1 구조로 설계하는 경우가 많다.

CQ 용량은 유한하므로, RQ를 만들 때 지정하는 `MaxOutstandingReceive` / `MaxOutstandingSend` 는 "이 소켓은 CQ 용량을 넘기지 않는다"는 보증이다. 이 한도는 RQ 생성 시점에 CQ 크기와 맞춰 검사되고 요청을 발행할 때도 다시 검사되며, CQ가 가득 차면 요청 발행이 `WSAENOBUFS` 로 실패한다(7.5절). `RIONotify` 를 쓰지 않고 `RIODequeueCompletion` 으로만 폴링할 CQ는 통지 구조체 자리에 `NULL` 을 넘겨 만든다(4절). IOCP 통지를 쓸 때 `Iocp.Overlapped` 는 유효한 `OVERLAPPED` 를 가리켜야 하고 CQ마다 전용 객체를 둔다. `Iocp.CompletionKey` 는 이 CQ의 통지를 다른 완료와 구별하는 데 쓸 수 있다.

3.3 배치 제출 — RIOSend / RIOReceive와 Notify

```
BOOL RIOSend(
    RIO_RQ SocketQueue,
    PRIO_BUF pData,
    ULONG DataBufferCount,
    DWORD Flags,        // RIO_MSG_DEFER 등
    PVOID RequestContext
);
```

`Flags` 에 `RIO_MSG_DEFER` 를 주면, 이 요청은 RQ에 큐잉되지만 즉시 실행되지 않을 수 있다(실행될 수도 있다). 지연된 송신은 같은 RQ에 `DEFER` 없는 `RIOSend` / `RIOSendEx` 가 호출될 때, 지연된 수신은 `DEFER` 없는 `RIOReceive` / `RIOReceiveEx` 가 호출될 때 함께 실행된다. 지연한 요청도 RQ의 동시 요청 한도에 포함된다. 마지막 요청만 `DEFER` 없이 걸면 그 요청이 다른 요청보다 훨씬 늦게 완료되므로, 문서는 쌓아 둔 요청을 `RIO_MSG_COMMIT_ONLY` 로 한꺼번에 제출하는 방식을 설명한다. `COMMIT_ONLY` 호출은 다른 플래그와 함께 쓸 수 없고 `pData` 와 `RequestContext` 는 `NULL` ,

`DataBufferCount`는 `0`이어야 하며, 같은 RQ에 대한 다른 `RIOSend`/`RIOReceive` 호출과 직렬화하지 않아도 된다. 매 패킷마다 시스템 콜을 발생시키는 대신, 여러 패킷을 모아 한 번에 제출하는 것이 핵심 아이디어다.

`RIONotify()`는 요청 제출 함수가 아니라, CQ에 완료가 쌓였을 때 통지를 받도록 CQ를 "무장"하는 함수다(4절 참고).

3.4 배치 완료 수거 — RIODequeueCompletion

```
RIORESULT results[256];
ULONG count = g_RIODequeueCompletion(cq, results, 256);

for (ULONG i = 0; i < count; ++i) {
    void* pContext = (void*)results[i].RequestContext;
    ULONG bytesTransferred = results[i].BytesTransferred;
    // RequestContext로 원래 요청(세션, 버퍼 슬롯 등) 식별 후 후처리
}
```

한 번의 호출로 최대 수백 건의 완료 결과를 배열로 받아올 수 있다. IOCP의 `GetQueuedCompletionStatus()`가 기본적으로 한 건씩 처리하는 것과 대비된다(`GetQueuedCompletionStatusEx`도 배치를 지원하지만, RIO는 애초에 이 배치 처리를 중심으로 설계됐다는 차이가 있다).

`RIORESULT`는 `Status`(`LONG`), `BytesTransferred`, `SocketContext`(RQ 생성 시 넘긴 값), `RequestContext`(요청 시 넘긴 값)로 이루어지며 호출 전에 초기화할 필요가 없다. 완료를 dequeue해야 비로소 시스템이 그 요청과 버퍼·버퍼 등록 사이의 연관과 할당량 차지를 해제한다. 그러므로 버퍼 등록 해제와 슬롯 반납은 해당 요청의 완료를 dequeue한 뒤에만 한다.

4. RIO와 IOCP의 관계

RIO의 CQ는 두 가지 모드로 쓸 수 있다.

- **폴링(Polling) 모드**: `RIONotify()`를 호출하지 않고, 워커 스레드가 주기적으로 `RIODequeueCompletion()`을 직접 호출해 완료 여부를 확인한다. 지연 시간을 최소화할 수 있지만 CPU를 계속 점유(busy-wait)하게 된다.
- **IOCP 연동 모드**: CQ 생성 시 `RIO_NOTIFICATION_COMPLETION`에 IOCP 핸들을 지정해두면, `RIONotify()` 호출 시 해당 CQ에 (`RIO_MSG_DONT_NOTIFY`를 주지 않은) 완료가 들어왔을 때 IOCP 완료 큐에 한 번의 통지가 전달된다. 호출 시점에 CQ가 이미 비어 있지 않으면 기다리지 않고 바로 통지된다. 통지의 `lpCompletionKey`는 `Iocp.CompletionKey`와 같고, `lpOverlapped`는 `Iocp.Overlapped`로 넘긴 객체다. 워커 스레드는 기존 IOCP 워커 루프(`GetQueuedCompletionStatus`)에서 이 신호를 받고, 그 시점에 `RIODequeueCompletion()`으로 실제 완료 목록을 배치로 꺼내온다.

실무에서는 대부분 후자를 쓴다. **RIO는 IOCP의 스케줄링/워커 스레드 모델은 그대로 재사용하면서, 데이터 경로(버퍼 검증, 제출·완료 처리)만 배치화해서 가볍게 만드는 확장**이라고 이해하는 것이 정확하다. RIO가 IOCP를 대체하는 것이 아니다.

전형적인 RIO 워커 루프:

```
void RioWorkerLoop(HANDLE hIOCP, RIO_CQ cq) {
    while (true) {
        DWORD bytesTransferred = 0;
        ULONG_PTR completionKey = 0;
        OVERLAPPED* pOverlapped = nullptr;

        BOOL ok = GetQueuedCompletionStatus(
            hIOCP, &bytesTransferred, &completionKey, &pOverlapped, INFINITE
        );
        if (!ok && pOverlapped == nullptr) {          // 완료 패킷을 꺼내지 못한
경우
            DWORD e = GetLastError();
            if (e == ERROR_ABANDONED_WAIT_0 || e == ERROR_INVALID_HANDLE)
break;  // 포트가 닫힘
            continue;
        }
        // (워커 종료 신호는 completionKey로 구분한다. 8.2절 참고)

        RIORESULT results[256];
        for (;;) {                                    // 큐가 빌 때까지 배치로 수거
            ULONG count = g_RIODequeueCompletion(cq, results, 256);
            if (count == RIO_CORRUPT_CQ) return;      // CQ가 손상됨: 폐기 대상,
재시도하지 않는다
            if (count == 0) break;
            for (ULONG i = 0; i < count; ++i)
                ProcessRioResult(results[i]);
        }

        g_RIONotify(cq); // 다음 완료를 다시 통지받기 위해 재무장
    }
}
```

한 번 통지를 받으면 큐가 완전히 빌 때까지 반복해서 배치 수거를 하고, 마지막에 다시 `RIONotify()`로 "재무장"하는 패턴이 일반적이다. 이 재무장을 빼먹으면 이후 완료가 쌓여도 IOCP로 통지가 오지 않는다. 마지막 `RIODequeueCompletion`이 0을 반환한 뒤 `RIONotify`를 호출하기 전에 완료가 들어왔더라도, `RIONotify`는 CQ가 이미 비어 있지 않으면 바로 통지하므로 완료가 유실되지 않는다. `RIODequeueCompletion`이 반환하는 `RIO_CORRUPT_CQ`는 항목 수가 아니라 손상 표시이므로 항목 수처럼 반복문에 쓰지 않는다.

5. 성능 이득이 나오는 지점과 나오지 않는 지점

5.1 이득이 큰 경우

- 패킷 크기가 작고 초당 처리 건수가 매우 많은 트래픽(UDP 기반 실시간 미디어, 금융 틱 데이터, 대규모 게임의 위치 동기화 패킷 등)
- 이미 IOCP + 최적화된 버퍼 풀로도 시스템 콜 비용이 프로파일링상 유의미한 병목으로 확인된 경우

5.2 이득이 크지 않거나 오히려 손해인 경우

- 동접/트래픽 규모가 크지 않은 일반적인 서비스: 오버헤드보다 구현·유지보수 복잡도 비용이 더 크다.
- 패킷 크기가 크고 빈도가 낮은 경우: 시스템 콜 비용 자체가 전체 처리 시간에서 차지하는 비중이 작다.
- TCP 스트림 위주에 순서 보장·재조립 로직이 이미 복잡한 경우: 여기에 버퍼 슬롯 관리까지 얹으면 복잡도가 급격히 늘어난다.

RIO 도입 전에는 반드시 IOCP만으로 프로파일링해서 병목이 정말 시스템 콜/버퍼 검증에 있는지 확인해야 한다. 실무에서 병목은 흔히 락 경합, 직렬화 비용, DB 접근 등 애플리케이션 로직에 있는 경우가 더 많다.

6. 구현 시 특히 주의할 점

RIO는 IOCP보다 직접 관리해야 하는 자원과 생명주기 규칙이 많다. 다음은 실무에서 자주 부딪히는 지점이다.

- **버퍼 슬롯 할당자의 동시성**: 여러 워커 스레드가 동시에 슬롯을 할당/반납하므로, 락프리 자료구조(예: Treiber 스택 기반 free list)나 세밀한 락 설계가 필요하다. ABA 문제, double-free 검출 같은 전형적인 동시성 이슈가 그대로 적용된다.
- **셧다운 순서**: RQ/CQ/등록 버퍼는 서로 의존 관계가 있다. 예를 들어 CQ를 먼저 파괴하고 아직 그 CQ를 참조하는 RQ에 I/O가 남아있으면 문제가 된다. "진행 중인 I/O가 0이 될 때까지 실제 자원 해제를 미룬다"는 원칙(IOCP에서도 동일하게 중요한 원칙)을 버퍼 등록 해제, CQ 파괴, RQ 파괴까지 전 계층에 일관되게 적용해야 한다.
- **RQ는 `closesocket()`으로 정리되지만 CQ와 버퍼 등록은 별도 해제 대상**: RQ에는 별도의 Close 함수가 없고, 소켓을 닫으면 RQ의 자원이 반환된다. `RIOCloseCompletionQueue()`와 `RIODeregisterBuffer()`는 명시적으로 의존 순서에 맞게 호출해야 한다. 닫힌 CQ로 들어오는 완료는 조용히 버려지고, 사용 중인 버퍼를 등록 해제하면 결과가 정의되지 않는다(문서는 이를 심각한 오류로 규정하며 Application Verifier로 탐지할 수 있다고 한다). 진행 중인 요청을 추적하는 책임은 애플리케이션에 있다.
- **재무장(RIONotify) 누락**: 위 워커 루프 예시처럼, 완료를 다 처리한 뒤 반드시 다시 `RIONotify()`를 호출해야 다음 완료 통지를 받을 수 있다. 이를 빼먹으면 트래픽이 멈춘 것처럼 보이는 디버깅하기 까다로운 버그가 된다.

- **CQ 크기와 요청 한도**: CQ 크기는 소속 RQ들의 `MaxOutstandingReceive + MaxOutstandingSend` 합 이상이어야 한다. 한도를 넘기면 요청 발행이 `WSAENOBUFS`로 실패한다. `RIO_CORRUPT_CQ`는 메모리 손상이나 RIO 함수 오용으로 CQ가 손상됐을 때 `RIODequeueCompletion`이 돌려주는 값이다.
- **동기화 규칙**: `RIO_CQ`와 `RIO_RQ`에 대한 접근은 내부적으로 동기화되지 않는다. 같은 CQ를 여러 스레드가 `RIODequeueCompletion` / `RIOResizeCompletionQueue`로 건드리거나 같은 소켓에 여러 스레드가 송신·수신을 발행하면 직접 동기화한다. 서로 다른 큐를 서로 다른 스레드가 쓰는 것은 잠금이 필요 없다.
- **등록 버퍼 사용 규칙**: 하나의 등록을 슬롯으로 나눠 쓰는 방식과 `RIOSend` 문서 문구의 관계는 3.1절을 참고한다. 최소한 같은 슬롯을 두 요청이 동시에 쓰지 않고, 요청이 끝나기 전에는 슬롯을 읽거나 쓰지 않는다.

7. API 레퍼런스 — 주요 함수 인터페이스

RIO 함수들은 일반 Winsock 함수와 달리 정적으로 링크되지 않는다.
`WSAIoctl(SIO_GET_MULTIPLE_EXTENSION_FUNCTION_POINTER)`로 함수 포인터 테이블을 직접 얻어와야 한다.

7.1 RIO 함수 포인터 테이블 로딩

```
RIO_EXTENSION_FUNCTION_TABLE g_RIO = { 0 };

GUID functionTableId = WSAID_MULTIPLE_RIO;
DWORD bytesReturned = 0;

WSAIoctl(
    dummySocket,                        // RIO 대상이 될 소켓(임시로 하나 생성해서 사
용)
    SIO_GET_MULTIPLE_EXTENSION_FUNCTION_POINTER,
    &functionTableId, sizeof(functionTableId),
    &g_RIO, sizeof(g_RIO),
    &bytesReturned,
    NULL, NULL
);
```

이렇게 채워진 `g_RIO` 구조체의 멤버(`g_RIO.RIORegisterBuffer`, `g_RIO.RIOSend` 등)를 이후 계속 호출하는 방식이다. 아래 각 함수는 이 테이블의 멤버로 존재한다. 문서는 이 호출에 쓰는 소켓의 조건을 따로 설명하지 않으며, 이 글은 RIO용 플래그로 만든 소켓(8.2절의 리슨 소켓)을 쓴다. `cbSize`는 구조체 크기를 나타내는 멤버이고 호출이 채워 준다. 테이블에는 이 글에서 다루지 않는 `RIOResizeRequestQueue`도 들어 있다(구조체 정의는 7.9절).

7.2 RIORegisterBuffer / RIODeregisterBuffer

```
RIO_BUFFERID RIORegisterBuffer(
    PCHAR DataBuffer,
    DWORD DataLength
);

VOID RIODeregisterBuffer(
    RIO_BUFFERID BufferId
);
```

파라미터	설명
`DataBuffer`	등록할 메모리 시작 주소. `VirtualAlloc` 등으로 미리 확보해둔 연속된 메모리
`DataLength`	등록할 길이(바이트). `0` 이면 `WSAEINVAL` 로 실패. 등록한 페이지는 물리 메모리에 고정되므로, 작은 버퍼를 따로 여러 개 등록하는 대신 큰 블록 하나를 등록한다(3.1절)
`BufferId`	등록 성공 시 반환되는 식별자. 이후 `RIO_BUF.BufferId` 에 사용. 실패 시 `RIO_INVALID_BUFFERID` 이며 `WSAGetLastError()` 로 원인을 확인한다(`WSAEFAULT` : 잘못된 `DataBuffer` 포인터, `WSAEINVAL` : 길이 0)

`RIODeregisterBuffer` 는 반환값이 없는(`VOID`) 함수다. 해당 버퍼를 아직 쓰는 I/O가 있을 때 호출하면 결과가 정의되지 않으며 문서는 이를 심각한 오류로 규정한다. 잘못된 `BufferId` 는 무시되고, 해제한 ID를 쓰는 이후 호출은 실패한다. 완료를 dequeue해야 시스템이 요청과 버퍼 등록의 연관을 풀기 때문에, 해제는 해당 버퍼를 쓴 모든 요청의 완료를 dequeue한 뒤에 한다.

7.3 RIOCreateCompletionQueue / RIOCloseCompletionQueue / RIOResizeCompletionQueue

```
RIO_CQ RIOCreateCompletionQueue(
    DWORD                         QueueSize,
    PRIO_NOTIFICATION_COMPLETION  NotificationCompletion
);

VOID RIOCloseCompletionQueue(RIO_CQ CQ);

BOOL RIOResizeCompletionQueue(
    RIO_CQ CQ,
    DWORD  QueueSize
);
```

파라미터	설명
`QueueSize`	이 CQ가 담을 수 있는 최대 완료 항목 수. 1 이상 `RIO_MAX_CQ_SIZE` 이하여야 하며 벗어나면 `WSAEINVAL`. 이 크기가 CQ에 연결할 수 있는 소켓 수와 요청 수를 제한하므로, 여러 RQ가 공유하는 만큼 예상 동시 처리량을 감안해 설정
`NotificationCompletion`	`RIO_NOTIFICATION_COMPLETION` 구조체. `Type`을 `RIO_EVENT_COMPLETION`(이벤트 기반: `Event.EventHandle`, `Event.NotifyReset`) 또는 `RIO_IOCP_COMPLETION`(IOCP 연동: `Iocp.IocpHandle`, `Iocp.CompletionKey`, `Iocp.Overlapped`)으로 설정한다. IOCP 연동 시 `Iocp.Overlapped`는 유효한 `OVERLAPPED`를 가리켜야 한다. `NULL`을 넘기면 통지 없이 폴링 전용 CQ가 된다
`CQ` (Resize)	크기를 조정할 기존 CQ

실패 시 `RIO_INVALID_CQ`를 반환하며 `WSAGetLastError()`로 원인을 확인한다(`WSAEFAULT`, `WSAEINVAL`, `WSAENOBUFS`). CQ가 가득 차면 이후 요청 발행이 `WSAENOBUFS`로 실패하므로, 트래픽 패턴에 맞춰 `QueueSize`를 산정하거나 필요 시 `RIOResizeCompletionQueue`로 동적으로 조정한다. `RIO_CORRUPT_CQ`는 가득 찬과는 별개로, 메모리 손상이나 RIO 함수 오용으로 CQ 상태가 깨졌을 때 `RIODequeueCompletion`이 돌려주는 값이다.

`RIOResizeCompletionQueue`는 큐를 키우거나 줄일 수 있고, 이미 들어 있는 완료는 새 큐로 복사된다. CQ에는 소속 RQ 수와 요청 수에 따른 최소 크기가 있어서 너무 작게 지정하면 실패하고 크기가 바뀌지 않는다(`WSAETOOMANYREFS`: 아직 CQ를 참조하는 작업이 많아 지금은 줄일 수 없음, `WSAENOBUFS`: 메모리 부족, `WSAEINVAL`: 잘못된 CQ 또는 크기 초과). 같은 CQ에 대한 `RIODequeueCompletion`과 `RIOResizeCompletionQueue` 호출은 서로 동기화한다.

`RIOCloseCompletionQueue`는 CQ를 무효로 표시하고 이후 들어올 완료를 조용히 버린다. 진행 중인 요청의 추적은 애플리케이션이 해야 하므로, 이 CQ를 쓰는 소켓의 I/O가 모두 끝난 뒤에 닫는다. 유효하지 않은 CQ를 넘기면 무시된다.

7.4 RIOCreateRequestQueue

```
RIO_RQ RIOCreateRequestQueue(
    SOCKET      Socket,
    ULONG       MaxOutstandingReceive,
    ULONG       MaxReceiveDataBuffers,
    ULONG       MaxOutstandingSend,
    ULONG       MaxSendDataBuffers,
    RIO_CQ      ReceiveCQ,
    RIO_CQ      SendCQ,
    PVOID       SocketContext
);
```

파라미터	설명
`Socket`	RIO용으로 만든 소켓. `WSASocket()`에 `WSA_FLAG_REGISTERED_IO`(`winsock2.h`에 정의된 매크로)를 주고 생성한다. `WSASocket` 문서의 `dwFlags` 목록에는 이 플래그가 나오지 않고, `RIO_RQ` 문서는 RIO 플래그로 만든 소켓이어야 한다고만 설명한다. `SOCK_RAW` 같은 지원하지 않는 소켓 종류는 `WSAEOPNOTSUPP`
`MaxOutstandingReceive`	이 RQ에 동시에 걸어둘 수 있는 최대 수신 요청 수. 수신·송신 한도가 모두 `0`이면 `WSAEINVAL`
`MaxReceiveDataBuffers`	수신 요청 1건당 최대 버퍼 수. 문서는 Windows 8 / Server 2012에서 이 값이 반드시 `1`이어야 한다고 명시한다. 따라서 요청마다 `RIO_BUF` 하나를 넘기며 scatter/gather는 쓰지 않는다
`MaxOutstandingSend` / `MaxSendDataBuffers`	송신 쪽 동일 개념
`ReceiveCQ` / `SendCQ`	이 RQ의 수신/송신 완료가 각각 어느 CQ로 갈지 지정. 같은 CQ를 공유해도 되고 분리해도 됨
`SocketContext`	이 RQ(소켓)를 식별할 사용자 정의 값. 이 RQ의 완료마다 `RIORESULT.SocketContext`로 돌아온다. 요청 단위 값은 `RIORESULT.RequestContext`로 따로 돌아온다

실패 시 `RIO_INVALID_RQ`를 반환하며 `WSAGetLastError()`로 원인을 확인한다(`WSAEINVAL`: CQ가 `RIO_INVALID_CQ`이거나 두 한도가 모두 0이거나 소켓이 초기화·종료 중, `WSAENOBUFS`: 메모리 부족 또는 네트워크 세션 한도 초과, `WSAENOTSOCK`, `WSAEOPNOTSUPP`).

RQ에는 별도의 명시적 Close 함수가 없다. 소켓을 `closesocket()` 으로 닫으면 RQ의 자원이 반환되고 그 RQ가 차지하던 CQ 한도도 다른 소켓이 쓸 수 있게 된다. 닫은 시점에 진행 중이던 수신 요청은 `WSA_OPERATION_ABORTED` 로 취소된다(`RIOReceive` 문서). 송신 요청의 취소 동작은 `RIOSend` 문서에 따로 적혀 있지 않으므로, 이 글은 소켓을 닫은 뒤에도 모든 요청의 완료가 dequeue될 때까지 컨텍스트와 슬롯을 유지한다. RQ와 CQ에 대한 접근은 동기화되지 않으며, 같은 소켓에 송신과 수신을 서로 다른 스레드가 발행할 때도 같은 RQ를 쓰므로 동기화가 필요하다.

7.5 RIOSend / RIOSendEx / RIOReceive / RIOReceiveEx

```
BOOL RIOSend(
    RIO_RQ SocketQueue,
    PRIO_BUF pData,
    ULONG DataBufferCount,
    DWORD Flags,
    PVOID RequestContext
);

BOOL RIOReceive(
    RIO_RQ SocketQueue,
    PRIO_BUF pData,
    ULONG DataBufferCount,
    DWORD Flags,
    PVOID RequestContext
);
```

파라미터	설명
`SocketQueue`	요청을 넣을 RQ
`pData`	`RIO_BUF` (등록된 버퍼 내 오프셋+길이). 한 요청에 하나만 넘긴다. bind된 UDP 소켓에서 페이로드가 필요 없으면 `NULL`
`DataBufferCount`	`pData` 가 `NULL` 이면 `0` , 아니면 `1`
`Flags`	비트 OR 조합. `RIO_MSG_DEFER` : 큐잉만 하고 즉시 실행하지 않을 수 있음(3.3절). `RIO_MSG_DONT_NOTIFY` : 이 요청의 완료는 `RIONotify` 통지를 일으키지 않음(완료는 CQ에 들어가므로 `RIODequeueCompletion` 으로는 수거할 수 있다). `RIO_MSG_COMMIT_ONLY` : 새 요청 없이 지연된 요청만 제출. 다른 플래그와 함께 쓸 수 없고 `pData` · `RequestContext` 는 `NULL` , `DataBufferCount` 는 `0` . `RIO_MSG_WAITALL` (수신 전용): 버퍼 구간이 가득 차거나 연결이 닫히거나 취소·오류가 날 때까지 완료하지 않음. UDP에서는 지원되지 않음

파라미터	설명
`RequestContext`	완료 시 `RIORESULT.RequestContext`로 그대로 돌아오는 값. 어떤 요청이었는지 식별하는 열쇠(IOCP의 `OVERLAPPED` 포인터와 같은 역할)

`RIOSendEx` / `RIOReceiveEx`는 여기에 UDP 목적지 주소(`pRemoteAddress`), 제어 정보(`pControlContext`, `pFlags` 등 확장 필드)를 추가로 받는 확장판으로, 주로 UDP 송수신에 사용한다.

```
BOOL RIOReceiveEx(
    RIO_RQ SocketQueue,
    PRIO_BUF pData,
    ULONG DataBufferCount,
    PRIO_BUF pLocalAddress,
    PRIO_BUF pRemoteAddress,
    PRIO_BUF pControlContext,
    PRIO_BUF pFlags,
    DWORD Flags,
    PVOID RequestContext
);
```

`pRemoteAddress`에 송신지 주소 정보를 함께 받아올 수 있어, `recvfrom()`과 동등한 기능을 RIO 경로로 수행할 수 있다.

`RIOSendEx`는 다음 시그니처를 가진다. `RIOReceiveEx`와 달리 `pLocalAddress`는 예약된 인자라 `NULL`이어야 하고, `pRemoteAddress`에는 목적지 주소를 담은 등록 버퍼 구간을 넘긴다(연결된 소켓이면 `NULL` 가능).

```
BOOL RIOSendEx(
    RIO_RQ SocketQueue,
    PRIO_BUF pData,
    ULONG DataBufferCount,
    PRIO_BUF pLocalAddress,    // 예약: NULL
    PRIO_BUF pRemoteAddress,   // 목적지 주소(SOCKADDR_INET 크기 이상). 연결된 소켓
이면 NULL 가능
    PRIO_BUF pControlContext,
    PRIO_BUF pFlags,
    DWORD Flags,
    PVOID RequestContext
);
```

`RIOReceiveEx`의 `pLocalAddress` / `pRemoteAddress`로 받는 주소는 `SOCKADDR_INET` 형식이므로 해당 `RIO_BUF`의 `Length`는 `sizeof(SOCKADDR_INET)` 이상이어야 한다. 주소 버퍼도 등록된 버퍼 안에 있어야 한다.

반환값: 성공 시 `TRUE`이며 완료는 이미 큐에 들어갔거나 나중에 들어간다. `FALSE`면 요청이 시작되지 않았고 완료도 오지 않는다. `WSAGetLastError()`로 원인을 확인한다.

오류	의미
`WSAEFAULT`	요청이 큐잉·실행되기 전에 `RIO_BUF`가 가리키는 버퍼 등록이 해제됐거나 버퍼가 해제됨
`WSAEINVAL`	`SocketQueue`가 유효하지 않음, 해당 방향에 쓸 수 없는 `Flags`, CQ의 무결성 손상 등
`WSAENOBUFS`	연결된 CQ가 가득 찼거나 해당 방향의 항목 수 0으로 만들어짐
`WSA_OPERATION_ABORTED`	(수신) 대기 중이던 요청이 소켓 닫힘(로컬·원격) 또는 `SIO_FLUSH`로 취소됨

버퍼 사용에 관한 문서의 규칙은 다음과 같다. 요청이 끝날 때까지 `RIO_BUF`가 가리키는 버퍼와 그 등록은 유효해야 한다. 진행 중인 수신의 버퍼는 완료 전에 읽거나 쓰지 않는다(다른 수신의 목적지나 송신의 원본으로 쓰는 것 포함). 송신 버퍼도 다른 송신이나 수신과 동시에 쓰지 않으며, 같은 `RIO_BUF`를 진행 중인 `RIOSend(Ex)`에 다시 넘기지 않는다. 등록 버퍼 전체에 대한 `RIOSend` 문서의 더 엄격한 문구는 3.1절에서 다룬다.

7.6 RIONotify

CQ에 완료가 쌓이면 통지(IOCP 완료 패킷 또는 이벤트)를 받을 수 있도록 CQ를 "무장"한다. 지연된(`RIO_MSG_DEFER`) 요청의 제출과는 무관하다.

```
INT RIONotify(RIO_CQ CQ);
```

파라미터	설명
`CQ`	재무장할 완료 큐

반환값: 성공 시 `ERROR_SUCCESS`(0). 실패하면 오류 코드를 직접 돌려준다. `WSAEINVAL`은 `RIO_INVALID_CQ` 같은 잘못된 CQ 등이고, `WSAEALREADY`는 이전 `RIONotify` 요청이 아직 완료되지 않았다는 뜻이다. 이미 무장된 상태에서 다시 호출했을 때 나오는 정상적인 상태 신호이므로 재무장 목적이라면 무시해도 된다.

CQ에 (`RIO_MSG_DONT_NOTIFY`가 아닌 요청의) 완료가 들어오면 통지가 한 번 발생하고, 호출 시점에 CQ가 이미 비어 있지 않으면 바로 통지된다. 이벤트 기반 CQ에서 `NotifyReset`이 0이 아니면 `RIONotify`가 이벤트를 자동으로 리셋하고, 스레드 풀 wait 객체를 쓸 때는 `SetThreadpoolWait`을 `RIONotify` 직후에 호출한다(먼저 호출하면 wait 콜백이 불필요하게 실행될 수 있다). IOCP 기반 CQ에서는 CQ마다 전용 `OVERLAPPED`를 쓰고, 필요하면 `Iocp.CompletionKey`로 CQ를 구별한다.

7.7 RIODequeueCompletion

CQ에 쌓인 완료 결과를 배치로 꺼내온다.

```
ULONG RIODequeueCompletion(
    RIO_CQ     CQ,
    PRIORESULT Array,
    ULONG      ArraySize
);
```

파라미터	설명
`CQ`	대상 완료 큐
`Array`	결과를 받을 `RIORESULT` 배열. `RIORESULT`는 `RequestContext`, `Status`, `BytesTransferred`, `SocketContext` 필드를 가짐
`ArraySize`	`Array`의 크기(한 번에 받을 최대 개수)

반환값: 실제로 꺼내온 항목 수. `0`이면 큐가 비어있다는 뜻이다. `RIORESULT`의 `Status`는 `LONG`, `SocketContext`와 `RequestContext`는 `ULONGLONG`이고, 요청 컨텍스트가 포인터이면 변환해서 쓴다. 완료를 dequeue하기 전에는 시스템이 해당 요청의 버퍼 연관과 할당량 차지를 풀지 않는다. 반환값이 `RIO_CORRUPT_CQ`(특수 값)이면 해당 CQ가 손상된 상태이므로 그 이후로는 재시도하지 말고 CQ를 폐기/재생성해야 한다.

호출 패턴은 보통 "큐가 빌 때까지 반복 호출 → 마지막에 `RIONotify()`로 재무장"이다(4절 예시 코드 참고).

7.8 정리 순서 요약

생성 순서	해제 순서(역순 권장)
① `RIORegisterBuffer`	④ `RIODeregisterBuffer`
② `RIOCreateCompletionQueue`	③ `RIOCloseCompletionQueue`
③ `RIOCreateRequestQueue` (소켓당)	② `closesocket()` (RQ도 함께 정리됨)
④ `RIOSend` / `RIOReceive` 반복	① outstanding I/O가 0이 될 때까지 대기 후 위 역순 진행

`RIODeregisterBuffer`는 그 버퍼를 쓴 모든 요청의 완료를 dequeue한 뒤에, `RIOCloseCompletionQueue`는 그 CQ를 쓰는 모든 소켓의 I/O가 끝난 뒤에 호출한다. CQ를 닫은 뒤 도착하는 완료는 조용히 버려진다. 이 표는 원칙을 보여주기 위한 단순화이며, 실제로는 "해당 자원을 참

조하는 진행 중인 I/O가 없음을 확인한 뒤에만 해제한다"는 규칙이 각 단계마다 개별적으로 성립해야 한다.

7.9 주요 구조체 정의와 공식 문서 범위

아래 정의는 Microsoft Learn의 해당 구조체 페이지에 나온 문법(syntax)이다.

```
typedef struct _RIO_BUF {
  RIO_BUFFERID BufferId;
  ULONG        Offset;
  ULONG        Length;
} RIO_BUF, *PRIO_BUF;

typedef struct _RIORESULT {
  LONG      Status;
  ULONG     BytesTransferred;
  ULONGLONG SocketContext;
  ULONGLONG RequestContext;
} RIORESULT, *PRIORESULT;

typedef struct _RIO_NOTIFICATION_COMPLETION {
  RIO_NOTIFICATION_COMPLETION_TYPE Type;
  union {
    struct {
      HANDLE EventHandle;
      BOOL   NotifyReset;
    } Event;
    struct {
      HANDLE IocpHandle;
      PVOID  CompletionKey;
      PVOID  Overlapped;
    } Iocp;
  };
} RIO_NOTIFICATION_COMPLETION, *PRIO_NOTIFICATION_COMPLETION;

typedef struct _RIO_EXTENSION_FUNCTION_TABLE {
  DWORD                          cbSize;
  LPFN_RIORECEIVE                RIOReceive;
  LPFN_RIORECEIVEEX              RIOReceiveEx;
  LPFN_RIOSEND                   RIOSend;
  LPFN_RIOSENDEX                 RIOSendEx;
  LPFN_RIOCLOSECOMPLETIONQUEUE   RIOCloseCompletionQueue;
  LPFN_RIOCREATECOMPLETIONQUEUE RIOCreateCompletionQueue;
  LPFN_RIOCREATEREQUESTQUEUE     RIOCreateRequestQueue;
  LPFN_RIODEQUEUECOMPLETION      RIODequeueCompletion;
  LPFN_RIODEREGISTERBUFFER       RIODeregisterBuffer;
```

```
  LPFN_RIONOTIFY                 RIONotify;
  LPFN_RIOREGISTERBUFFER         RIORegisterBuffer;
  LPFN_RIORESIZECOMPLETIONQUEUE RIOResizeCompletionQueue;
  LPFN_RIORESIZEREQUESTQUEUE     RIOResizeRequestQueue;
} RIO_EXTENSION_FUNCTION_TABLE, *PRIO_EXTENSION_FUNCTION_TABLE;
```

이 절의 함수 설명은 Microsoft Learn의 `RIOCreateRequestQueue`, `RIOCreateCompletionQueue`, `RIONotify`, `RIODequeueCompletion`, `RIOSend`, `RIOSendEx`, `RIOReceive`, `RIOReceiveEx`, `RIORegisterBuffer`, `RIODeregisterBuffer`, `RIOCloseCompletionQueue`, `RIOResizeCompletionQueue` 페이지와 `RIO_NOTIFICATION_COMPLETION`, `RIORESULT`, `RIO_BUF`, `RIO_EXTENSION_FUNCTION_TABLE`, `RIO_RQ` 페이지를 기준으로 했다. 이 페이지들에는 실행 가능한 예제 코드가 없고 문법과 설명만 있다(`RIOReceiveEx` 페이지에는 `WSACMSGHDR`의 잘린 조각만 있다). 그래서 8절의 예제는 공식 예제의 전재가 아니라 이 글이 직접 작성한 것이다. 일부 페이지에는 설명이 비어 있는 인자(`RIOSend`의 `RequestContext`, `RIOReceiveEx`의 `pFlags`)와 서로 어긋나는 서술(3.1절)이 있으므로, 구현 전에 원문 페이지를 확인한다.

8. 사용법과 사용 예시

8.1 사용 순서

단계	작업	사용 API
1	Winsock 초기화	`WSAStartup`
2	RIO 함수 테이블 로딩	`WSAIoctl(SIO_GET_MULTIPLE_EXTENSION_FUNCTION_POINTER)`
3	큰 메모리 블록 확보 후 한 번 등록, 슬롯 할당자 구성	`VirtualAlloc`, `RIORegisterBuffer`
4	워커마다 IOCP와 CQ 생성, CQ 무장	`CreateIoCompletionPort`, `RIOCreateCompletionQueue`, `RIONotify`
5	워커 스레드 시작	`GetQueuedCompletionStatus` → `RIODequeueCompletion` → `RIONotify`
6	연결 수락 후 소켓당 RQ 생성, 슬롯 할당	`RIOCreateRequestQueue`
7	첫 Receive 요청	`RIOReceive`
8	Receive 완료 시: 데이터 처리 → Receive 재요청	`RIOReceive`

단계	작업	사용 API
9	Send 요청: 송신 큐에 적재, 진행 중인 Send가 없을 때만 발행	`RIOSend`
10	종료: 새 연결 중단 → 모든 세션 close → 세션 소멸(진행 중 I/O 0) 대기 → 워커 종료 신호 → CQ 닫기 → 버퍼 등록 해제	`closesocket`, `PostQueuedCompletionStatus`, `RIOCloseCompletionQueue`, `RIODeregisterBuffer`

설계 원칙은 다음과 같다.

- **소켓은 `WSA_FLAG_REGISTERED_IO`로 생성한다.** RQ는 이 플래그로 만든 소켓에 만든다. `AcceptEx`를 쓴다면 미리 만들어 두는 소켓에 플래그를 준다. 이 글의 예시는 `accept()`로 받은 소켓이 리슨 소켓의 플래그를 이어받는다고 가정하는데, 공식 문서에는 이 동작이 명시되어 있지 않다. 이어받지 않는 환경이면 `RIOCreateRequestQueue`가 실패하므로(예시는 오류를 출력한다) `AcceptEx` 방식으로 바꾼다.
- **CQ 1개를 워커 스레드 1개가 전담한다.** 이 글의 예시는 워커마다 IOCP도 따로 두어 "CQ ↔ 스레드" 1:1을 보장한다. 같은 CQ에 대한 `RIODequeueCompletion`은 동기화 없이 동시에 호출하지 않는다.
- **CQ 크기는 그 CQ를 쓰는 모든 RQ의 `MaxOutstandingReceive + MaxOutstandingSend` 합 이상으로 잡는다.** 한도는 RQ 생성 시점과 요청 발행 시점에 검사되며, 가득 찬 CQ에 대한 요청 발행은 `WSAENOBUFS`로 실패한다. `RIO_CORRUPT_CQ`는 `RIODequeueCompletion`이 CQ 손상을 알리는 값이다.
- **같은 RQ에 대한 `RIOSend` / `RIOReceive` 호출은 동기화한다.** 예시는 세션당 뮤텍스를 사용한다.
- 컨텍스트가 세션의 `shared_ptr`를 들고, 완료 시 워커가 꺼내 소유한다. 슬롯은 세션 소멸자에서 반납하므로, 진행 중인 I/O가 0이 된 뒤에만 반납된다.
- **세션 레지스트리로 종료를 보장한다.** 살아 있는 세션을 `weak_ptr`로 추적해 종료 시 모두 닫고, 전부 소멸할 때까지 기다린 뒤에 워커 종료 신호를 보낸다. 신호를 먼저 보내면 워커가 빠져나간 뒤 도착하는 완료를 처리할 주체가 없다.
- **등록 버퍼 하나를 슬롯으로 나눠 쓴다.** 이 방식과 `RIOSend` 문서 문구의 관계는 3.1절의 주의를 참고한다.

8.2 TCP 에코 서버 전체 예시

빌드 시 `ws2_32.lib`를 링크한다.

공통 정의

```
#include <winsock2.h>
#include <mswsock.h>
#include <ws2tcpip.h>
#include <chrono>
```

```
#include <cstdio>
#include <memory>
#include <string>
#include <mutex>
#include <atomic>
#include <thread>
#include <unordered_map>
#include <vector>
#include <algorithm>

constexpr DWORD     SLOT_SIZE            = 4096;
constexpr int       MAX_SESSIONS_PER_WORKER = 1000;
constexpr ULONG_PTR KEY_SHUTDOWN         = static_cast<ULONG_PTR>(-1);
constexpr DWORD     INVALID_OFFSET       = 0xFFFFFFFF;

RIO_EXTENSION_FUNCTION_TABLE g_rio{};
RIO_BUFFERID                 g_bufId = RIO_INVALID_BUFFERID;
char*                        g_pool  = nullptr;
std::atomic<bool>            g_running{true};

inline RIO_BUF MakeBuf(DWORD offset, ULONG length) {
    RIO_BUF b;
    b.BufferId = g_bufId;
    b.Offset   = offset;
    b.Length   = length;
    return b;
}
```

슬롯 할당자

세션당 Receive 슬롯 1개, Send 슬롯 1개를 쓴다. 예시는 뮤텍스 기반 free list이며, 실서비스에서는 락프리 구조로 교체한다(6절).

```
class SlotPool {
public:
    void Init(DWORD slotCount) {
        std::lock_guard<std::mutex> lk(_m);
        _free.reserve(slotCount);
        for (DWORD i = 0; i < slotCount; ++i)
            _free.push_back(i * SLOT_SIZE);
    }
    bool Acquire(DWORD& offset) {
        std::lock_guard<std::mutex> lk(_m);
        if (_free.empty()) return false;
        offset = _free.back();
        _free.pop_back();
        return true;
```

```
    }
    void Release(DWORD offset) {
        std::lock_guard<std::mutex> lk(_m);
        _free.push_back(offset);
    }
private:
    std::mutex         _m;
    std::vector<DWORD> _free;
};

SlotPool g_slots;
```

RIO 함수 테이블 로딩

```
bool LoadRio(SOCKET s) {
    GUID  id    = WSAID_MULTIPLE_RIO;
    DWORD bytes = 0;
    return WSAIoctl(s, SIO_GET_MULTIPLE_EXTENSION_FUNCTION_POINTER,
                    &id, sizeof(id), &g_rio, sizeof(g_rio),
                    &bytes, nullptr, nullptr) == 0;
}
```

워커와 세션

`SessionRegistry`는 살아 있는 세션을 `weak_ptr`로 추적한다. 종료 시 모든 세션을 닫고 전부 소멸할 때까지 기다리는 데 쓰며, 세션 수명에는 관여하지 않는다. `~RioSession`에서 항목을 제거하므로 `make_shared`가 만든 메모리 블록이 레지스트리 때문에 남지 않는다.

```
struct Worker {
    HANDLE            iocp = nullptr;
    RIO_CQ            cq   = RIO_INVALID_CQ;
    OVERLAPPED        ov{};                    // CQ 전용 OVERLAPPED (통지 패킷
에 실려 돌아옴)
    std::atomic<int>  sessionCount{0};
    std::thread       th;
};

enum class RioOp { Recv, Send };

class RioSession;

struct RioContext {
    RioOp                          op;
    std::shared_ptr<RioSession>  owner;        // 요청이 진행되는 동안 세션을 붙잡는
다
};
```

```
// 살아 있는 세션 목록. 종료 시 모든 세션을 닫고, 전부 소멸할 때까지 기다리는 데 쓴다.
// weak_ptr만 들고 있으므로 세션 수명에는 관여하지 않는다.
struct SessionRegistry {
    void   Add(const std::shared_ptr<RioSession>& s);
    void   Remove(RioSession* s);              // ~RioSession에서 호출
    void   CloseAll();
    size_t Size();

    std::mutex                                                  _m;
    std::unordered_map<RioSession*, std::weak_ptr<RioSession>> _map;
};

SessionRegistry g_registry;

class RioSession : public std::enable_shared_from_this<RioSession> {
public:
    RioSession(SOCKET s, Worker* w) : _sock(s), _worker(w) {
        _recvCtx.op = RioOp::Recv;
        _sendCtx.op = RioOp::Send;
    }

    ~RioSession() {
        Close();
        if (_recvOff != INVALID_OFFSET) g_slots.Release(_recvOff);
        if (_sendOff != INVALID_OFFSET) g_slots.Release(_sendOff);
        _worker->sessionCount.fetch_sub(1);
        g_registry.Remove(this);               // 모든 자원을 돌려준 뒤 마지막에 제
거
    }

    bool Init() {
        if (!g_slots.Acquire(_recvOff)) { _recvOff = INVALID_OFFSET; return
false; }
        if (!g_slots.Acquire(_sendOff)) { _sendOff = INVALID_OFFSET; return
false; }

        // 동시 요청 수: Receive 1, Send 1, 요청당 버퍼 1
        _rq = g_rio.RIOCreateRequestQueue(_sock, 1, 1, 1, 1, _worker->cq,
_worker->cq, nullptr);
        if (_rq == RIO_INVALID_RQ) {
            std::fprintf(stderr, "RIOCreateRequestQueue failed: %d\n",
WSAGetLastError());
            return false;
        }
        return true;
```

```
    }

    void PostRecv() {
        std::lock_guard<std::mutex> lk(_lock);
        PostRecvLocked();
    }

    void OnRecv(ULONG bytes) {
        Send(g_pool + _recvOff, bytes);        // 에코: 수신 슬롯 내용을 송신 큐로
복사
        PostRecv();                            // 복사가 끝났으므로 슬롯 재사용 가능
    }

    void Send(const char* data, size_t len) {
        std::lock_guard<std::mutex> lk(_lock);
        if (_closed) return;
        _pending.append(data, len);
        if (!_sending) {
            _sending = true;
            PostNextSendLocked();
        }
    }

    void OnSend(ULONG bytes) {
        std::lock_guard<std::mutex> lk(_lock);
        if (_closed) { _sending = false; return; }
        if (bytes < _sendLen) {                // 부분 전송: 남은 구간을 이어서 전
송
            _sendOffset += bytes;
            _sendLen    -= bytes;
            IssueSendLocked();
            return;
        }
        if (_pending.empty()) { _sending = false; return; }
        PostNextSendLocked();
    }

    void Close() {
        if (!_closed.exchange(true))
            closesocket(_sock);                // 진행 중인 요청은 에러로 완료되어
owner가 풀린다
    }

private:
    void PostRecvLocked() {
        if (_closed) return;
```

```
        _recvCtx.owner = shared_from_this();
        RIO_BUF b = MakeBuf(_recvOff, SLOT_SIZE);
        if (!g_rio.RIOReceive(_rq, &b, 1, 0, &_recvCtx)) {
            _recvCtx.owner.reset();            // 완료가 오지 않으므로 직접 해제
            Close();
        }
    }

    void PostNextSendLocked() {
        size_t n = std::min<size_t>(_pending.size(), SLOT_SIZE);
        memcpy(g_pool + _sendOff, _pending.data(), n);
        _pending.erase(0, n);
        _sendOffset = _sendOff;
        _sendLen    = static_cast<ULONG>(n);
        IssueSendLocked();
    }

    void IssueSendLocked() {
        _sendCtx.owner = shared_from_this();
        RIO_BUF b = MakeBuf(_sendOffset, _sendLen);
        if (!g_rio.RIOSend(_rq, &b, 1, 0, &_sendCtx)) {
            _sendCtx.owner.reset();
            _sending = false;
            Close();
        }
    }

    SOCKET             _sock;
    Worker*            _worker;
    RIO_RQ             _rq = RIO_INVALID_RQ;
    DWORD              _recvOff = INVALID_OFFSET, _sendOff = INVALID_OFFSET;

    std::mutex         _lock;                  // RQ 호출과 송신 상태 보호
    RioContext         _recvCtx, _sendCtx;
    std::string        _pending;               // 아직 슬롯에 담기지 않은 송신 데이
터
    bool               _sending = false;
    DWORD              _sendOffset = 0;
    ULONG              _sendLen = 0;
    std::atomic<bool>  _closed{false};
};

void SessionRegistry::Add(const std::shared_ptr<RioSession>& s) {
    std::lock_guard<std::mutex> lk(_m);
    _map[s.get()] = s;
}
```

```
void SessionRegistry::Remove(RioSession* s) {
    std::lock_guard<std::mutex> lk(_m);
    _map.erase(s);
}

void SessionRegistry::CloseAll() {
    std::vector<std::shared_ptr<RioSession>> alive;
    {
        std::lock_guard<std::mutex> lk(_m);
        for (auto& kv : _map)
            if (auto s = kv.second.lock()) alive.push_back(std::move(s));
    }
    // 잠금 밖에서 닫는다. 마지막 참조가 여기서 풀리면 ~RioSession이 Remove()로 _m을 잡는다.
    for (auto& s : alive) s->Close();
}

size_t SessionRegistry::Size() {
    std::lock_guard<std::mutex> lk(_m);
    return _map.size();
}
```

워커 스레드

```
void Dispatch(const RIORESULT& r) {
    auto* ctx  = reinterpret_cast<RioContext*>(static_cast<ULONG_PTR>(r.RequestContext));
    auto  self = std::move(ctx->owner);        // 이 스코프 동안 세션 생존 보장

    if (r.Status != 0) { self->Close(); return; }

    if (ctx->op == RioOp::Recv) {
        if (r.BytesTransferred == 0) { self->Close(); return; }   // 상대의 정상 종료
        self->OnRecv(r.BytesTransferred);
    } else {
        self->OnSend(r.BytesTransferred);
    }
}

void WorkerLoop(Worker* w) {
    RIORESULT results[256];

    for (;;) {
        DWORD       bytes = 0;
```

```
        ULONG_PTR   key   = 0;
        OVERLAPPED* ov    = nullptr;

        BOOL ok = GetQueuedCompletionStatus(w->iocp, &bytes, &key, &ov,
INFINITE);
        if (!ok && ov == nullptr) {                    // 완료 패킷을 꺼내지 못한
경우
            const DWORD e = GetLastError();
            if (e == ERROR_ABANDONED_WAIT_0 || e == ERROR_INVALID_HANDLE)
break;  // 포트가 닫힘
            continue;
        }
        if (key == KEY_SHUTDOWN) break;

        // CQ가 빌 때까지 배치로 수거
        bool corrupt = false;
        for (;;) {
            ULONG n = g_rio.RIODequeueCompletion(w->cq, results, 256);
            if (n == RIO_CORRUPT_CQ) { corrupt = true; break; }
            if (n == 0) break;
            for (ULONG i = 0; i < n; ++i)
                Dispatch(results[i]);
        }
        if (corrupt) break;                            // CQ 폐기 대상: 로그 후
워커 종료

        g_rio.RIONotify(w->cq);                        // 재무장
    }
}
```

워커 생성

```
bool StartWorker(Worker& w) {
    w.iocp = CreateIoCompletionPort(INVALID_HANDLE_VALUE, nullptr, 0, 1);
    if (!w.iocp) return false;

    RIO_NOTIFICATION_COMPLETION nc{};
    nc.Type                   = RIO_IOCP_COMPLETION;
    nc.Iocp.IocpHandle        = w.iocp;
    nc.Iocp.CompletionKey     = nullptr;              // 워커마다 포트가 따로라
구분이 필요 없다
    nc.Iocp.Overlapped        = &w.ov;                // CQ 전용 OVERLAPPED
(유효한 객체여야 함)

    // CQ 크기: 이 CQ를 쓰는 모든 RQ의 (Recv + Send) 동시 요청 수 합 이상
    w.cq = g_rio.RIOCreateCompletionQueue(MAX_SESSIONS_PER_WORKER * 2, &nc);
    if (w.cq == RIO_INVALID_CQ) return false;

    g_rio.RIONotify(w.cq);                            // 최초 무장
    w.th = std::thread(WorkerLoop, &w);
    return true;
}
```

main

```
int main() {
    WSADATA wsa;
    if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0) return 1;

    SOCKET listenSock = WSASocket(AF_INET, SOCK_STREAM, IPPROTO_TCP, nullptr,
0,
                                  WSA_FLAG_REGISTERED_IO);
    if (listenSock == INVALID_SOCKET || !LoadRio(listenSock)) {
        std::fprintf(stderr, "RIO unavailable: %d\n", WSAGetLastError());
        return 1;
    }

    const unsigned workerCount = std::max(1u,
std::thread::hardware_concurrency());

    // 큰 블록 하나를 등록하고 슬롯으로 분할
    const DWORD slotCount = workerCount * MAX_SESSIONS_PER_WORKER * 2;
    const DWORD poolSize  = slotCount * SLOT_SIZE;
    g_pool = static_cast<char*>(VirtualAlloc(nullptr, poolSize,
                                             MEM_COMMIT | MEM_RESERVE,
PAGE_READWRITE));
    if (!g_pool) return 1;
    g_bufId = g_rio.RIORegisterBuffer(g_pool, poolSize);
```

```
    if (g_bufId == RIO_INVALID_BUFFERID) {
        std::fprintf(stderr, "RIORegisterBuffer failed: %d\n",
WSAGetLastError());
        return 1;
    }
    g_slots.Init(slotCount);

    std::vector<std::unique_ptr<Worker>> workers;
    for (unsigned i = 0; i < workerCount; ++i) {
        workers.push_back(std::make_unique<Worker>());
        if (!StartWorker(*workers.back())) return 1;
    }

    sockaddr_in addr{};
    addr.sin_family      = AF_INET;
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
    addr.sin_port        = htons(9000);
    if (bind(listenSock, reinterpret_cast<sockaddr*>(&addr), sizeof(addr)) !=
0 ||
        listen(listenSock, SOMAXCONN) != 0) {
        std::fprintf(stderr, "bind/listen failed: %d\n", WSAGetLastError());
        return 1;
    }

    // 수락 스레드: 세션을 워커에 라운드로빈으로 배정.
    // 리슨 소켓을 다른 스레드에서 닫지 않도록, select로 짧게 기다리며 종료 플래그를 확인한
다.
    std::thread acceptor([&] {
        unsigned next = 0;
        while (g_running) {
            fd_set rd;
            FD_ZERO(&rd);
            FD_SET(listenSock, &rd);
            timeval tv{0, 100 * 1000};
            const int r = select(0, &rd, nullptr, nullptr, &tv);
            if (r == SOCKET_ERROR) break;
            if (r == 0) continue;

            SOCKET c = accept(listenSock, nullptr, nullptr);
            if (c == INVALID_SOCKET) continue;

            Worker* w = workers[next++ % workerCount].get();
            if (w->sessionCount.fetch_add(1) >= MAX_SESSIONS_PER_WORKER) {
                w->sessionCount.fetch_sub(1);          // CQ/슬롯 용량 초과 — 거
절
                closesocket(c);
```

```
                continue;
            }

            auto session = std::make_shared<RioSession>(c, w);   // 소멸자가
sessionCount를 감소
            g_registry.Add(session);
            if (!session->Init()) { session->Close(); continue; }
            session->PostRecv();
        }
    });

    getchar();                                          // 서버 가동 중

    // ---- 종료 ----
    // 1) 새 연결을 막는다.
    g_running = false;
    acceptor.join();
    closesocket(listenSock);

    // 2) 모든 세션을 닫고, 진행 중인 I/O가 전부 회수되어 세션이 소멸할 때까지 기다린다.
    g_registry.CloseAll();
    const auto deadline = std::chrono::steady_clock::now() +
std::chrono::seconds(10);
    while (g_registry.Size() != 0 && std::chrono::steady_clock::now() <
deadline)
        std::this_thread::sleep_for(std::chrono::milliseconds(10));

    const size_t remaining = g_registry.Size();
    std::printf("[RIO] shutdown: remaining sessions = %zu\n", remaining);
    if (remaining != 0) {
        // 진행 중인 I/O가 남아 있으면 버퍼 등록 해제는 정의되지 않은 동작이다. 해제하지 않
고 끝낸다.
        std::fprintf(stderr, "WARNING: shutdown invariant violated; skipping
resource release.\n");
        return 2;
    }

    // 3) 이제 워커에 종료 신호를 보낸다.
    for (auto& w : workers)
        PostQueuedCompletionStatus(w->iocp, 0, KEY_SHUTDOWN, nullptr);
    for (auto& w : workers) {
        w->th.join();
        g_rio.RIOCloseCompletionQueue(w->cq);
        CloseHandle(w->iocp);
    }
    g_rio.RIODeregisterBuffer(g_bufId);
```

```
    VirtualFree(g_pool, 0, MEM_RELEASE);
    WSACleanup();
    return 0;
}
```

동작 확인은 `telnet 127.0.0.1 9000`으로 입력한 문자열이 그대로 돌아오는지 보면 된다.

종료 순서는 main의 주석과 같다. 새 연결을 막고(수락 스레드는 리슨 소켓을 다른 스레드에서 닫지 않도록 `select`로 짧게 기다리며 종료 플래그를 확인한다), 모든 세션을 닫고, 세션이 전부 소멸할 때까지 기다린 뒤에 워커에 종료 신호를 보내고, CQ를 닫고, 버퍼 등록을 해제한다. 대기 시간 제한(10초)을 넘기면 진행 중인 I/O가 남은 것이므로, 사용 중인 버퍼의 등록 해제가 정의되지 않은 동작이라는 점을 고려해 자원을 해제하지 않고 종료한다. 7.8절의 해제 순서는 모든 세션이 소멸해 진행 중인 I/O가 없는 상태를 전제로 한다.

8.3 배치 제출 예시 — 한 RQ에 여러 패킷을 한 번의 제출로

RQ 생성 시 `MaxOutstandingSend`를 N 이상으로 잡고, 모든 요청을 `RIO_MSG_DEFER`로 큐잉한 뒤 `RIO_MSG_COMMIT_ONLY`로 한꺼번에 제출한다. 마지막 요청만 `DEFER` 없이 걸어도 쌓인 요청이 함께 제출되지만, 그 마지막 요청이 다른 요청보다 훨씬 늦게 완료되므로 문서는 `COMMIT_ONLY`를 권한다.

```
// RQ 생성 시: MaxOutstandingSend >= n
bool SendBatch(RIO_RQ rq, RioContext* ctxs, RIO_BUF* bufs, int n) {
    for (int i = 0; i < n; ++i)
        if (!g_rio.RIOSend(rq, &bufs[i], 1, RIO_MSG_DEFER, &ctxs[i]))   // 큐
잉만
            return false;                                                 //
실패한 요청은 완료가 오지 않는다
    return g_rio.RIOSend(rq, nullptr, 0, RIO_MSG_COMMIT_ONLY, nullptr);  //
쌓인 요청을 한 번에 제출
}
```

Receive도 같은 방식이며, 송신 큐와 수신 큐의 지연 요청은 각각 `RIOSend` / `RIOReceive` (`COMMIT_ONLY` 포함)로 제출한다. `COMMIT_ONLY` 호출은 같은 RQ의 다른 `RIOSend` / `RIOReceive` 호출과 직렬화하지 않아도 된다. 요청마다 `RequestContext`가 완료 시점까지 유효해야 하므로, 예시처럼 요청 수만큼 컨텍스트를 별도로 둔다. 지연한 요청도 RQ의 동시 요청 한도와 CQ 용량을 차지한다. `RIONotify`는 이 제출과 무관하다.

8.4 폴링 모드 예시

IOCP 연동 없이 전용 스레드가 CQ를 계속 확인한다. 지연 시간은 가장 짧지만 코어 하나를 점유한다.

```
// 통지 구조체를 NULL로 주면 폴링 전용 CQ가 된다.
RIO_CQ cq = g_rio.RIOCreateCompletionQueue(queueSize, nullptr);

void PollingLoop(RIO_CQ cq, std::atomic<bool>& running) {
    RIORESULT results[256];
    while (running) {
        ULONG n = g_rio.RIODequeueCompletion(cq, results, 256);
        if (n == RIO_CORRUPT_CQ) break;
        if (n == 0) { _mm_pause(); continue; }         // <intrin.h>
        for (ULONG i = 0; i < n; ++i)
            Dispatch(results[i]);
    }
}
```

폴링 모드에서는 `RIONotify`를 호출하지 않는다. 폴링 CQ도 한 스레드가 전담하거나 `RIODequeueCompletion` 호출을 동기화한다.

8.5 UDP 예시 — RIOReceiveEx / RIOSendEx

주소 정보도 등록된 버퍼 안에 있어야 하고, `RIO_BUF`의 `Length`는 `sizeof(SOCKADDR_INET)` 이상이어야 한다. 슬롯 하나(`SLOT_UDP`)를 `[데이터 | 송신지 주소]`로 나눠 쓴다. `RIO_MSG_WAITALL`은 UDP에서 지원되지 않는다.

```
constexpr DWORD DATA_SIZE = 2048;
constexpr DWORD SLOT_UDP  = DATA_SIZE + sizeof(SOCKADDR_INET);

SOCKET udp = WSASocket(AF_INET, SOCK_DGRAM, IPPROTO_UDP, nullptr, 0,
WSA_FLAG_REGISTERED_IO);
// bind(udp, ...) 후 RQ 생성
RIO_RQ rq = g_rio.RIOCreateRequestQueue(udp, 8, 1, 8, 1, cq, cq, nullptr);

// 수신 요청: 데이터와 송신지 주소를 같은 슬롯에 받는다
RIO_BUF data   = MakeBuf(slotOff,             DATA_SIZE);
RIO_BUF remote = MakeBuf(slotOff + DATA_SIZE, sizeof(SOCKADDR_INET));
g_rio.RIOReceiveEx(rq, &data, 1, nullptr, &remote, nullptr, nullptr, 0,
&recvCtx);

// 수신 완료(bytes) 후: 받은 주소 그대로 에코 송신
RIO_BUF reply = MakeBuf(slotOff, bytes);
g_rio.RIOSendEx(rq, &reply, 1, nullptr, &remote, nullptr, nullptr, 0,
&sendCtx);
```

`RIOSendEx`의 송신 완료 전까지 슬롯(데이터와 주소 영역 모두)을 반납하거나 재사용하지 않는다. 단일 소켓에 Receive를 여러 개 걸어두려면 `MaxOutstandingReceive`와 CQ 크기를 그만큼 늘리고, 요청마다 별도 슬롯을 쓴다.

8.6 사용 시 점검 항목

- 소켓을 `WSA_FLAG_REGISTERED_IO`로 만들었는가.
- CQ 크기가 소속 RQ들의 `MaxOutstandingReceive + MaxOutstandingSend` 합 이상인가. 가득 차면 요청 발행이 `WSAENOBUFS`로 실패하는 경로를 처리하는가.
- CQ 생성 후 최초 `RIONotify`를 호출했는가. 완료를 다 수거한 뒤에는 매번 재무장하는가.
- `RIODequeueCompletion`이 `RIO_CORRUPT_CQ`를 반환하는 경우를 처리하는가.
- 같은 CQ에 대한 `RIODequeueCompletion`, 같은 RQ에 대한 `RIOSend`/`RIOReceive`를 동기화 없이 동시에 호출하지 않는가.
- 요청 발행이 즉시 실패(`FALSE`)하면 완료가 오지 않는다. 이 경우 컨텍스트의 `owner`를 직접 해제하는가.
- 슬롯은 해당 슬롯을 쓰는 모든 요청이 완료된 뒤에만 반납하는가.
- `RequestContext`가 가리키는 객체가 완료 시점까지 유효한가.
- 종료 순서가 "새 연결 중단 → 세션 close → 진행 중 I/O 0 확인(세션 소멸 대기) → 워커 종료 신호 → CQ 닫기 → 버퍼 등록 해제"인가. 대기가 끝나지 않으면 자원을 해제하지 않고 끝내는가.
- 워커 루프가 `GetQueuedCompletionStatus` 실패(포트 닫힘 등)와 `RIO_CORRUPT_CQ`를 처리하는가.
- `accept()`로 받은 소켓에서 `RIOCreateRequestQueue`가 성공하는지 대상 환경에서 확인했는가(8.1절의 가정은 문서에 명시되어 있지 않다).
- 하나의 등록 버퍼를 슬롯으로 나눠 쓰는 설계가 3.1절의 문서 불일치를 이해한 상태에서 검증되었는가.

9. 마무리

RIO는 IOCP의 완료 통지 모델을 그대로 유지하면서, 버퍼 사전 등록과 배치 제출·수거로 시스템 콜·버퍼 검증 오버헤드를 극한까지 줄이기 위한 특수 목적 확장이다. 대부분의 서버는 IOCP만으로 충분하며, RIO는 실측으로 병목이 확인된 초고빈도·초고처리량 트래픽 상황에서 선택적으로 고려하는 것이 합리적이다. 도입한다면 버퍼 슬롯 관리와 셧다운 순서를 처음부터 원칙 있게 설계하는 것이 가장 중요하다.